

Proyecto Sistemas Operativos

Liz Cartaya Salabarría C211
Maya Ramón Castellanos C212

Universidad de La Habana
Facultad de Matemática y Computación

1. Introducción

El presente proyecto se enmarca en la asignatura Sistemas Operativos de segundo año de la carrera Ciencia de la Computación. Su objetivo principal es el aprendizaje práctico mediante la implementación, modificación y análisis de componentes reales del sistema operativo, abordando aspectos como la gestión de procesos, la planificación de la CPU, la gestión de memoria, los mecanismos de concurrencia a través de hilos y el acceso al sistema de archivos mediante llamadas al sistema.

Como plataforma de trabajo se emplea MINIX 3. Su naturaleza modular y la disponibilidad de su código fuente permiten modificar, extender y analizar componentes internos reales, como el planificador de procesos, las bibliotecas de hilos y el sistema de entrada/salida.

El trabajo se estructuró en varios hitos que integran progresivamente los conceptos mencionados. Primero, personalizamos el mensaje de bienvenida del sistema. Luego, abordamos la depuración de un bug en la implementación de `pthread_mutex_trylock`, lo que requirió comprender la relación entre la capa de compatibilidad `pthread` y la implementación nativa `mthread` de MINIX. Implementamos el comando `tree`, que recorre recursivamente el sistema de archivos utilizando llamadas al sistema como `opendir`, `readdir` y `lstat`. Finalmente, modificamos el planificador de MINIX para penalizar procesos con uso intensivo de CPU.

En las siguientes secciones se documentan detalladamente las decisiones de diseño, los cambios realizados en el código, las dificultades encontradas y las pruebas de validación ejecutadas para cada uno de estos componentes.

2. Desarrollo y resultados por componente

2.1. Instalación y configuración de VirtualBox y MINIX

2.1.1. Software utilizado

El proyecto se desarrolló sobre una laptop física con Ubuntu 24.04 LTS (sistema anfitrión). Sobre esta, se instaló Oracle VirtualBox 7.2.6 y se descargó la imagen ISO de MINIX 3.4.0 desde el sitio oficial.

Creamos una máquina virtual a la que nombramos Project Minix con MINIX 3 como sistema operativo invitado, con una memoria Ram de 2048 megabytes y con tres CPUs.

2.1.2. Instalación y Configuración

Instalamos, seguimos la configuración sugerida y quitamos el ISO de instalación.

Instalamos los binarios sugeridos y para eso primero actualizamos la lista de paquetes:

```
1 minix# pkgin update
```

Instalamos git para poder trabajar con repositorios:

```
1 minix# pkgin install git-base
```

Y siguiendo la documentación oficial utilizamos el siguiente comando para instalar lo mínimo necesario:

```
1 minix# pkgin_sets
```

Procedemos a clonar el repositorio para poder obtener el código del sistema. Para ello antes realizamos los comandos:

```
1 minix# cd /usr/src/  
2 minix# pwd
```

y ya ubicados en esta carpeta clonamos el repositorio:

```
1 minix# git clone https://github.com/MRCastellanos321/minix
```

Tuvimos un problema con el certificado de SSL que resolvemos de la siguiente manera:

```
1 minix# git -c http.sslVerify=false clone https://github.com/  
MRCastellanos321/minix
```

Mientras se clonaba tuvimos un problema que impedía que el repositorio se clonara correctamente. Esto se debía a que el tamaño virtual de nuestra máquina era de solo 2048 megabytes y resultaba insuficiente. Para eso modificamos el espacio virtual a unos 8 GB y volvimos a clonar el repositorio, esta vez con éxito.

De esta forma ya quedó todo listo para trabajar, hacer cambios y compilar.

2.1.3. Reenvío de puertos

Para poder acceder a la máquina virtual desde la terminal de Ubuntu de forma cómoda, se configuró el reenvío de puertos en VirtualBox, redirigiendo el tráfico del puerto 2222 del host al puerto 22 del invitado (MINIX). De esta forma, es posible conectarse por SSH sin necesidad de abrir puertos adicionales en el sistema anfitrión.

Desde la interfaz gráfica de VirtualBox fuimos a: Settings → Network → Adapter 1 (NAT) → Port Forwarding y creamos una nueva regla con nombre: ssh, protocol: tcp, Host Port: 2222 y Guest Port: 22.

Actualizamos el gestor de paquetes y se instalaron las herramientas necesarias para la compilación y edición:

```
1 minix# pkgin update  
2 minix# pkgin install openssh
```

Configuramos el servidor SSH dentro de MINIX para permitir conexiones remotas:

```
1 minix# cp /usr/pkg/etc/rc.d/sshd /etc/rc.d/  
2 minix# printf 'sshd=YES\n' >> /etc/rc.conf  
3 minix# printf 'PermitRootLogin yes\n' >> /usr/pkg/etc/ssh/  
sshd_config
```

Creamos una contraseña de seguridad para permitir la conexión entre ambas terminales: `minix# passwd`

Iniciamos el servidor SSH en modo depuración:

```
1 minix# /usr/pkg/sbin/sshd -d
```

Finalmente, desde la terminal de Ubuntu, nos conectamos a MINIX:

```
1 ssh root@127.0.0.1 -p 2222
```

De esta forma podemos trabajar de manera más agradable desde la terminal de Ubuntu.

2.1.4. Carpeta compartida

Para facilitar la transferencia de archivos entre MINIX y Ubuntu, se configuró una carpeta compartida en VirtualBox.

Desde Ubuntu creamos la carpeta: `mkdir ~/minix_compartida`

Desde la interfaz gráfica de VirtualBox fuimos a: Machine → Settings → Shared Folders y creamos una nueva carpeta compartida. Le agregamos la ruta de la carpeta en Ubuntu, de nombre le pusimos `compartida` marcamos las opciones `Automontar` y `Hacer permanente`.

Por último, montamos la carpeta dentro de MINIX:

```
1 minix# mkdir -p /mnt/compartida
2 minix# mount -t vbfs -o share=compartida none /mnt/compartida
```

2.2. Personalización del mensaje de bienvenida

Para completar esta tarea debimos modificar el mensaje de bienvenida tanto en el sistema MINIX de la máquina virtual como en el repositorio de git. Siguiendo las indicaciones de la orientación del proyecto modificamos el archivo `/etc/motd` en MINIX utilizando el editor de texto `vi`.

```
1 minix# vi /etc/motd
```

Eliminamos el texto contenido en el archivo utilizando, dentro de `vi`, el comando `dd` que borra una línea completa. Luego escribimos nuestro mensaje personalizado:

Bienvenido a MINIX 3

Facultad de Matematica y Computacion
Universidad de La Habana

Guardamos el mensaje con el comando `:wq` y salimos del editor.

Reiniciamos la máquina virtual escribiendo `reboot` en la terminal y una vez que iniciada podemos ver el mensaje anterior.

Una vez modificado el mensaje en la máquina virtual para que el cambio se mantuviera en futuras compilaciones del sistema, replicamos la modificación en `/usr/src/minix/etc/motd`, que es el archivo fuente del mensaje de bienvenida.

```
1 minix# cp /etc/motd /usr/src/minix/etc/motd
```

Luego recompilamos el sistema y copiamos el archivo fuente recompilado a la carpeta compartida:

```
1 minix# cp /etc/motd /mnt/compartida/
```

desde la cual copiamos dicho archivo a la carpeta del repositorio en Ubuntu, para así subir los cambios.

2.3. Depuración de un bug en pthread

Creamos un archivo test_mutex.c con el programa de prueba de referencia que tenemos en la orientación del proyecto y al compilar y ejecutar este test:

```
1 minix# cd /root
2 minix# cc -o test_mutex test_mutex.c -lmthread
3 minix# ./test_mutex
```

notamos que el programa se queda colgado desde la primera llamada.

Para encontrar el error localizamos el archivo pthread_compat.c:

```
1 minix# find /usr/src -name "pthread_compat.c" 2>/dev/null
```

Revisamos el código siguiendo la ruta antes encontrada:

```
1 minix# nano /usr/src/minix/minix/lib/libmthread/pthread_compat.c
```

Allí pudimos ver el código de la función pthread_mutex_trylock:

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
4         mthread_mutex_init(mutex, NULL);
5     }
6
7     return pthread_mutex_trylock(mutex);
8 }
```

y notamos que la función luego de crear un mutex, si no existe, se llama a sí misma recursivamente creando una recursión infinita y haciendo que el programa quede colgado. En su lugar debería llamar al método mthread_mutex_trylock.

Esto se debe a que pthread es un estándar de programación para manejo de hilos (threads) en sistemas UNIX, pero MINIX no implementa pthread de forma nativa, sino que tiene su propio sistema de hilos llamado mthread (Minix Threads). Para permitir la ejecución de programas escritos para el estándar pthread, MINIX incorpora una capa de compatibilidad en el archivo pthread_compat.c. Esta capa funciona como traductor, expone las funciones pthread* y redirige las llamadas a las funciones equivalentes de mthread*.

Cambiamos el código:

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     if (PTHREAD_MUTEX_INITIALIZER == *mutex) {
4         mthread_mutex_init(mutex, NULL);
5     }
6 -     return pthread_mutex_trylock(mutex);}
```

```
7 +     return mthread_mutex_trylock(mutex);}  
8 }
```

Compilamos solo la biblioteca libmthread e instalamos la biblioteca compilada en el sistema (/usr/lib) para que los cambios reemplazaran versiones antiguas:

```
1 minix# cd /usr/src/minix/lib/libmthread  
2 minix# make  
3 minix# make install
```

Volvimos a compilar y ejecutar el test que creamos:

```
1 minix# cd /root  
2 minix# cc -o test_mutex test_mutex.c -lmthread  
3 minix# ./test_mutex
```

y pudimos ver que obtuvimos las respuestas esperadas:

```
first trylock: 0 (OK)  
second trylock: 11 (Resource deadlock avoided)  
unlock: 0 (OK)  
destroy: 0 (OK)  
PASS
```

Para confirmar el significado del valor 11, consultamos el archivo /usr/include/sys/errno.h, donde comprobamos que en esta versión de MINIX la constante EDEADLK (Resource deadlock avoided) está definida con el valor 11.

2.4. Implementación del comando tree

Para crear el comando tree, accedemos a la carpeta commands en el código fuente de minix y creamos un nuevo directorio para contener el mismo. Creamos un archivo donde programaremos el comando en c y dentro comienza la implementación.

Diseño de algoritmo e Indentación

La idea del programa es recorrer recursivamente a partir de la ruta dada (o "." en caso de que no haya parámetro para el tree) cada carpeta y listarlas a ellas y sus archivos. El main que va a ser el punto de entrada nos permite ver la ruta decidida por el usuario a través del argc y el argv. La función `abrirRuta` es la que se llamará a sí misma de forma recursiva. La recursividad permite que en cada ruta donde se analicen las carpetas y archivos, sea posible llamarla sobre el nuevo directorio de las carpetas, y llevar a través de la variable requerida como parámetro "nivel" un tamaño de indentación para imprimir cada nombre de forma espaciada de acuerdo a su profundidad.

Elegimos recursividad en lugar de un enfoque iterativo con pila porque refleja de forma natural la estructura jerárquica del sistema de archivos, cada directorio puede contener subdirectorios, y la función simplemente se invoca a sí misma sobre cada uno. La profundidad típica de un sistema de archivos no supera el límite que pueda causar un stackoverflow. Además, la recursión simplifica el manejo de la indentación: el nivel se pasa como parámetro y se incrementa en cada llamada.

Llamadas al sistema

Utilizamos las funciones que importamos con `#include <dirent.h>` y `<sys/stat.h>`:

`closedir()`; recibe un `*DIR` y cierra el directorio abierto que apunta

`opendir()`; abre un directorio (recibe un texto ruta y devuelve un `*DIR`)

`readdir()`; recibe un `*DIR` y devuelve un `*struct dirent`

`lstat()`; recibe Ruta (texto) y puntero a `struct stat`, devuelve 0 al éxito

y -1 al fallo. Lee los metadatos de un archivo (tipo, tamaño, permisos, propietario, ...) y los guarda en una estructura `struct stat`.

Prevención de ciclos

Se tiene en cuenta como requerido en la orientación comprobar los enlaces simbólicos, los cuales pueden causar bucles infinitos. Un enlace simbólico es un puntero virtual a otro directorio que en casos especiales puede causar un ciclo interminable (si apunta a la carpeta que lo contiene o un ancestro, esta se va a abrir por la función y encontrarse de nuevo con el enlace simbólico, recorriendo la misma ruta una y otra vez, por ejemplo). Los detalles de su manejo se encuentran a continuación en código donde vamos a ver el uso de la macro `S_ISLNK(rutacomprobacion.st_mode)` que va a evaluar como true en caso de que sea un enlace simbólico y nos va a permitir evitarlos.

Código

Comenzamos con main. Si la cuenta de argumentos es `>1`, pasamos la ruta indicada por el usuario como parámetro para el primer llamado de la función `abrirRuta`, de lo contrario es el directorio actual.

```
1 char *primeraRuta = ".";
2 if(argc > 1)
3 {
4     primeraRuta = argv[1];
5 }
6 printf("%s\n", primeraRuta);
7 abrirRuta(primeraRuta, 0);
```

También debemos manejar errores (no hay permisos para el archivo, etc). Por ello comprobamos que el intento de obtener el `*DIR` a través de `opendir()` no haya fallado:

```
1 DIR *directorioabierto = opendir(ruta);
2 if (directorioabierto == NULL)
3 {
4     printf("Error\n");
5     return;
6 }
7 struct dirent *nombreActual;
8 while ((nombreActual = readdir(directorioabierto)) != NULL)
9 {
10     char *nombre = nombreActual->d_name;
```

```

11 // ...
12 }

```

En este caso `readdir` devuelve un puntero al struct `dirent`, `nombreActual`, llamado así por conveniencia, y es lo que da acceso a propiedades como `d_name` que vamos a usar para imprimir los nombres de archivos y carpetas en la terminal.

En la función se pasa a comprobar los archivos de inicio "." (pues son los especiales normalmente invisibles del sistema) para no imprimir los nombres de estos.

Tenemos en cuenta los enlaces simbólicos. Para prevenir el caso de un bucle infinito:

```

1 char rutaCompleta[1024];
2 snprintf(rutaCompleta, sizeof(rutaCompleta), "%s/%s", ruta,
   nombre);
3
4 struct stat rutacomprobacion;
5 if (lstat(rutaCompleta, &rutacomprobacion) == -1)
6 {
7     continue;
8 }
9 if (S_ISLNK(rutacomprobacion.st_mode))
10 {
11     printf("%s\n", nombre);
12     continue;
13 }

```

En este fragmento de código concatenamos mediante `snprintf` (que no imprime en pantalla sino que guarda en string) para guardar en `rutaCompleta` la unión de `ruta` y `nombre`, permitiendo así pasarle a `lstat` la ruta completa que requiere de parámetro. Declaramos la variable tipo struct `stat` `rutacomprobacion`, que `lstat` rellena con los metadatos de `rutaCompleta`, para así poder usar la macro `S_ISLNK(rutacomprobacion.st_mode)` que va a evaluar como true en caso de que sea un enlace simbólico, permitiéndonos imprimir el nombre y continuar a la siguiente iteración del ciclo mientras ignoramos el enlace.

Por último:

```

1 DIR *prueba = opendir(rutaCompleta);
2 if (prueba != NULL)
3 {
4     closedir(prueba);
5     printf("%s\n", nombre);
6     abrirRuta(rutaCompleta, nivel + 1);
7 }
8 else
9 {
10     printf("%s\n", nombre);
11 }

```

Aquí se tiene en cuenta si lo obtenido en `rutaCompleta` se trata de una carpeta o archivo (si es una carpeta, no dará NULL el if), esto es puesto que en el primer caso hay que adentrarse recursivamente en la carpeta tras imprimir su nombre, mediante un llamado a `abrirRuta` usando como parámetro este directorio. Aumentamos nivel en 1 para imprimir a más profundidad, y concluimos con `closedir()`.

El árbol también requiere de un archivo Makefile, el que usamos para compilar el sistema con el nuevo comando y ponerlo en funcionamiento.

Ejemplos de ejecución

A continuación las imágenes resultados de llamar al comando tree para el directorio actual (fig #1), para una ruta relativa (fig #2) y para una absoluta (fig #3), y una cuarta para mostrar el manejo de error (fig #4). En las tres primeras podemos ver que el `enlace_a_carpeta` (simbólico) no fue seguido hacia la carpeta `EnlaceApuntaAqui`, y que los niveles fueron impresos con indentación. La cuarta imagen (fig #4) muestra como al hacer un login con un usuario distinto a la raíz e intentar elegir como ruta un directorio sin permisos (TestError en este caso) la pantalla imprime un error.

```
minix# cd TreePruebas
minix# tree
.
CarpetaNivel1
  CarpetaNivel2
    CarpetaNivel3(Copy)
      archivoNivel4.md
      enlace_a_carpeta
      EnlaceApuntaAqui
        archivoEnCarpetaApuntada.md
      archivoNivel3.md
    archivoNivel2(Copy).md
    archivoNivel2.md
  archivoNivel1.md
minix# _
```

Figura 1: Tree sin ruta

```
minix# pwd
/usr/src/minix/TreePruebas
minix# tree CarpetaNivel1/CarpetaNivel2
CarpetaNivel1/CarpetaNivel2
CarpetaNivel3(Copy)
  archivoNivel4.md
  enlace_a_carpeta
EnlaceApuntaAqui
  archivoEnCarpetaApuntada.md
archivoNivel3.md
minix#
```

Figura 2: Tree con ruta relativa

```

minix# tree /usr/src/minix/TreePruebas
/usr/src/minix/TreePruebas
CarpetaNivel1
  CarpetaNivel2
    CarpetaNivel3(Copy)
      archivoNivel4.md
      enlace_a_carpeta
    EnlaceApuntaAqui
      archivoEnCarpetaApuntada.md
      archivoNivel3.md
    archivoNivel2(Copy).md
    archivoNivel2.md
  archivoNivel1.md

```

Figura 3: Tree con ruta absoluta

```

minix$ cd /usr/src/minix
minix$ cd TreePruebas
minix$ cd CarpetaNivel1
minix$ tree TestError
TestError
Error
minix$

```

Figura 4: Manejo de error

2.5. Comparación

Comparamos con el comando `tree` de Linux. Este tiene una salida similar, muestra la estructura de directorios con indentación, diferencia archivos y carpetas, y no sigue enlaces simbólicos por defecto. Nuestra implementación cumple con la indentación y el evitar los enlaces. Es una versión simplificada sin opciones de formato visual como colores, pero sigue siendo funcional y cumple con lo solicitado en la orientación del proyecto.

2.6. Penalización por uso intensivo de CPU

2.6.1. Análisis teórico del algoritmo e introducción

Los diferentes procesos del sistema están en constante demanda de CPU, a menudo compitiendo por la misma, y es tarea del SO seguir una estrategia para la distribución óptima de ella para su mejor funcionamiento.

La planificación de procesos es el mecanismo del SO que decide qué proceso usa la CPU en cada momento y durante cuánto tiempo. En un sistema multiprogramado hay varios procesos en memoria listos para ejecutarse, pero CPU limitada. Sin planificación, no habría orden: un proceso podría acaparar la CPU indefinidamente mientras otros esperan eternamente. La planificación crea la ilusión de que todos los procesos avanzan simultáneamente, aunque en realidad se turnan.

MINIX 3.0 sin modificar sigue la estrategia de Round Robin: Cada proceso recibe un quantum fijo (`USER_QUANTUM = 200ms` como vemos en los archivos del código fuente), y cuando este se agota, se llama a `do_noquantum()` (función del `scheduler.c`) y el proceso pasa al final de la cola. Es decir, funciona con los procesos en una sola cola circular, reciben el mismo quantum de tiempo, y rotan en orden. La idea de esta estrategia es cambiar de

proceso al que se le permite ejecutarse cada cierto intervalo de tiempo, por lo que logra un buen tiempo de respuesta, ya que ninguno de ellos espera demasiado para empezar a ejecutarse. Sin embargo, esta estrategia trata igual a todos los procesos, tiene mal tiempo de retorno puesto que un proceso corto se sigue rotando con otros más largos, y permite que ciertos procesos acaparen CPU.

La modificación implementada por nosotras para su mejoramiento consiste en una versión simplificada de MLFQ(Multi-level Feedback Queue) visto en conferencia, y que tiene en cuenta los problemas del Round Robin. Esta estrategia consiste en:

- Ajusta prioridades según el comportamiento.
- Procesos CPU-Bound : reciben menor prioridad.
- Procesos interactivos o bloqueantes: reciben mayor prioridad.
- Incluye "priority boost" para evitar inanición (la inanición es cuando un proceso espera eternamente por CPU y no logra ejecutarse)

Análisis previo

Como pide la orientación, debemos modificar la forma en que la CPU se administra, implementando una penalización en los programas por su uso excesivo. Analizamos los archivos con los que vamos a trabajar. Para ello accedimos a `minix/servers/sched` en el código fuente. Aquí se encuentra `sched.c`, el que va a contener las funciones necesarias para el trabajo, `do_noquantum()`, `balance_queues()`, `do_start_scheduling()` y `do_stop_scheduling()`, y `schedproc.h` que también necesitaremos modificar. Vemos la existencia de la variable `priority` y `max_priority` para cada proceso. Funciona de esta forma: una mayor `priority` indica que la prioridad del sistema para este proceso es menor, o sea que al aumentar "priority" para un proceso en realidad estamos disminuyendo su prioridad para que no acapare CPU, usando los `quantums` como medida.

Vamos a explicar el funcionamiento de las principales funciones a modificar y que originalmente están implementando el Round Robin:

`do_noquantum()` es la función que es llamada cuando un proceso agotó su `quantum`. Como explicamos antes, cada proceso recibe un `quantum`, que es un tiempo de CPU máximo otorgado por el sistema. Cuando el timer del kernel detecta que este llegó a cero, envía un mensaje al scheduler, el cual entonces llama a esta función. Ella vuelve a poner el proceso en la cola de listos.

La función `balance_queues()` por otra parte se va a encargar de balancear. Anteriormente, cada 5 segundos recorría todos los procesos del sistema y si su prioridad era peor que `max_priority`, la subía en un nivel.

`do_start_scheduling()` se llama cuando un nuevo proceso está listo para ser planificado. Originalmente asigna prioridad, `quantum` y CPU al proceso.

`do_stop_scheduling()` se llama cuando un proceso se bloquea voluntariamente (por ejemplo, al esperar E/S o un semáforo). Originalmente esta función libera el lugar de planificación del proceso.

Diseño de política

Como explicamos anteriormente, debemos modificar el código para implementar MLFQ. Especificamos a continuación las decisiones de cómo pensamos hacer funcionar las penalizaciones y el balance:

Tras la conclusión de las modificaciones, las prioridades se ajustan según el comportamiento, penalizando procesos que consumen mucha CPU. Llevamos la cuenta de la siguiente forma: cada $N=3$ quantums consumidos, un proceso es penalizado en `priority++`. Elegimos $N=3$ como sugerido en la orientación porque es buen balance entre una penalización demasiado agresiva y una que tarda demasiado en aplicarse. Un proceso interactivo o bloqueante no será penalizado.

Observemos ahora la línea existente en `sched.c` :

```
1 #define BALANCE\_TIMEOUT 5 /* how often to balance queues in
   seconds */
```

Y en `int_scheduling`:

```
1 balance\_timeout = BALANCE\_TIMEOUT * sys\_hz();
2
3 sys\_setalarm(balance\_timeout, 0);
```

De aquí sabemos que el `balance_queues()` se llama cada 5s.

Por eso implementamos nuestra estrategia de forma tal que las ventanas de intervalo aprovechan la estructura ya existente del llamado a `balance_queues()` para tener los cinco segundos cada los cuales esta función es llamada como su tamaño, así no es necesario definir nuevos contadores y se modifica el código en lo mínimo. Esta elección permite aprovechar las herramientas que ya fueron dadas por la estrategia base. En esta misma función si al concluir ese tiempo (la función es llamada) el proceso tiene `quantum_count = 0` y no ha alcanzado su `max_priority`, entonces recupera su prioridad (`priority--`), puesto que no se ha estado comportando como un proceso abarcador de CPU, e independientemente, `quantum_count` vuelve a ser 0. De esta forma aseguramos que un proceso interactivo o uno CPU-Bound que cambió su comportamiento recupere su lugar en la cola, y evitamos inactivación.

Cambios Realizados

Iniciamos modificando el archivo `schedproc.h` para añadir la variable `quantum_count`, que utilizaremos para la condición de N quantums (en este caso elegimos 3 como explicado anteriormente). Ahora `quantum_count` es accesible desde `rmp` en el código.

Vamos al archivo `sched.c` donde están las funciones que vamos a modificar. Primeramente, el objetivo es aplicar una penalización gradual a los programas que alcancen los 3 quantums o más. Iniciamos en `do_noquantum()` añadiendo esta línea tras obtener el puntero al `rmp`:

```
1 rmp = &schedproc[proc_nr_n];
2 rmp->quantum_count++; // aumentamos el contador
```

Luego toca aplicar la penalización. Para ello, cuando `rmp->quantum_count` es mayor o igual que 3 aumentamos en +1 su `priority`. Comprobamos también que el aumento no supera el `MIN_USER_Q` definido por el sistema y reseteamos la cuenta de quantums a 0 una vez concluido esto. Queda modificado el código de `do_noquantum()` con este segmento añadido:

```
1 if (rmp->quantum_count >= 3) // alcanzo el umbral de penalizacion
2 {
3     if (rmp->priority < MIN_USER_Q) // no supera prioridad minima
4     {
```

```

5     rmp->priority++; //penalizar
6 }
7 rmp->quantum_count = 0; //reiniciar contador tras penalizar
8 }

```

Ahora, `balance_queues()`. Tras la modificación, si un proceso tiene su cuenta de quantum en 0 (es decir que no se comportó como un proceso intensivo durante esa ventana) y su `priority` es mayor (peor) que `max_priority`, entonces disminuimos `priority` (o sea aumentamos su prioridad). Además, al concluir la ventana temporal se reinicia la cuenta de quantum a 0. Así usamos el mecanismo ya existente de temporización. Queda este segmento modificado:

```

1 if (rmp->flags & IN_USE) ////para procesos activos
2 {
3     if (rmp->quantum_count == 0 //Si no fue CPU-bound en la
4         ventana de tiempo
5         && rmp->priority > rmp->max_priority) //y su prioridad es
6         peor que la maxima
7     {
8         rmp->priority -= 1; //mejorar su prioridad
9         schedule_process_local(rmp); //avisa al kernel de la
10        nueva prioridad
11    }
12 }
13 rmp->quantum_count = 0; //fin de ventana, reiniciar

```

2.6.2. Validación experimental

Para ambos casos creamos un código en c en la carpeta pruebas. El objetivo es comprobar que el MLFQ se esté comportando correctamente para el caso de un proceso CPU-Bound y para un proceso interactivo.

Para evaluar el primero, elegimos como programa un ciclo infinito:

```

1 int main()
2 {
3     while(1)
4     {
5     }
6     return 0;
7 }

```

Luego, nos servimos de los comandos `cc -o cpuBound cpuBound.c` y `./cpuBound`, el primero compila el programa usando `cc` y el segundo lo ejecuta. En una segunda terminal, el comando `'top ()` (con `| grep cpuBound` para encontrar la línea) nos permitió ver el progreso de la penalización, puesto que el loop infinito agota quantums sin terminar nunca y por ello es penalizado hasta `priority 15`. También añadimos un `printf` temporal en el código `sched.c` (en `do_noquantum()` porque cada vez que es llamada es cuando ocurre la modificación que queremos ver) para observar mejor el aumento de la penalización en el tiempo sin necesidad de los comandos anteriores y obtener la imagen a continuación. Mostrado en la fig.5 "Proceso CPU-bound"

Para evaluar el segundo caso, proceso interactivo o bloqueante:

```

1 int main(void)
2 {
3     int i;
4     for (i = 0; i < 10; i++)
5     {
6         sleep(1);
7     }
8     return 0;
9 }

```

Ejecutamos la segunda prueba de la misma forma. Vemos en la terminal que el proceso bloqueante no recibe ninguna penalización a su prioridad, porque usa ‘sleep()’ cada cierto tiempo para bloquearse y por tanto no es categorizado como un proceso CPU-Bound o intensivo para la CPU, ya que la está cediendo cada vez que se bloquea. Observemos la terminal en la imagen fig.6 ”Proceso interactivo”

Imágenes(capturas de terminal en minix que muestran la evolución de la prioridad para ambos casos):

```

minix# cc -o cpuBound cpuBound.c
minix# ./cpuBound
SCHED: proc 223 quantum_count=1 priority=7
SCHED: proc 223 quantum_count=2 priority=8
SCHED: proc 223 quantum_count=3 priority=9
SCHED: proc 223 quantum_count=1 priority=11
SCHED: proc 223 quantum_count=2 priority=12
SCHED: proc 223 quantum_count=3 priority=13
SCHED: proc 223 quantum_count=1 priority=15
SCHED: proc 223 quantum_count=1 priority=15
SCHED: proc 223 quantum_count=2 priority=15
SCHED: proc 223 quantum_count=3 priority=15
SCHED: proc 223 quantum_count=1 priority=15
SCHED: proc 223 quantum_count=2 priority=15
SCHED: proc 223 quantum_count=3 priority=15
SCHED: proc 223 quantum_count=1 priority=15
SCHED: proc 223 quantum_count=2 priority=15
SCHED: proc 223 quantum_count=3 priority=15

```

Figura 5: Proceso CPU-bound

```

minix# ./interactivo &
minix# while true; do ps -o pid,pri,comm | grep interactivo; sleep 2; d
746 7 ./interactivo
746 7 ./interactivo
746 7 ./interactivo
746 7 ./interactivo
746 7 ./interactivo

```

Figura 6: Proceso interactivo

3. Resultados globales y discusión

Concluido el proyecto todas las componentes y cambios implementados funcionan satisfactoriamente, por lo que se ha cumplido la meta, y no encontramos diferencias al resultado esperado. Se superaron las dificultades técnicas iniciales de espacio y conectividad para preparar el ambiente de trabajo. Aprendimos más acerca de la estructuración interna de un sistema operativo, la programación de comandos de terminal a bajo nivel, el manejo y distribución de cpu para los programas según distintas estrategias y lo que hacen algunas de las funciones esenciales para su diseño, nos familiarizamos con el uso

de comandos de terminal nuevos y conocidos a lo largo de la realización del proyecto, así como con varios aspectos del lenguaje c.

A lo largo del desarrollo del proyecto se enfrentaron varios obstáculos técnicos que requirieron investigación:

1. Configuración del entorno virtual: Inicialmente, el tamaño de la máquina virtual (2 GB) era insuficiente para clonar el repositorio de MINIX. Tras diagnosticar el error relacionado con el espacio en disco, se amplió a 8 GB, resolviendo el problema. También se solucionaron problemas de certificados SSL y se configuró el reenvío de puertos y carpetas compartidas para facilitar el trabajo.

2. Depuración del bug en pthread: Identificar la recursión infinita en `pthread_mutex_trylock` requirió localizar el archivo correcto en el árbol de fuentes (`pthread_compat.c`) y comprender la relación entre las dos implementaciones de hilos. La corrección fue sencilla una vez comprendido el error, pero el proceso de depuración enseñó a navegar por el código del sistema.

3. Programación del tree: A pesar de la dificultad inicial del lenguaje y funciones nuevas utilizadas, se realizó el código exitosamente, eligiendo recursión, con manejo de error para la ruta inicial, y previniendo seguir bucles infinitos a través de los enlaces simbólicos.

4. Modificación del planificador o scheduler: Implementamos MLFQ con un contador de quants por proceso, modificando `schedproc.h` y cuatro funciones en `sched.c`. El mayor desafío fue comprender el papel de las funciones para aplicar sus respectivos cambios y lograr que la penalización solo se aplicara tras consumir 3 quants y que la recuperación ocurriera en ventanas de 5 segundos mediante `balance_queues()`. Las pruebas con un proceso CPU-bound y uno interactivo demostraron el correcto funcionamiento de lo implementado.

4. Conclusiones

El desarrollo del presente proyecto nos ha permitido cumplir con los objetivos propuestos, abordando principios fundamentales de los sistemas operativos como la gestión de procesos, la planificación de la CPU, la concurrencia mediante hilos y el acceso al sistema de archivos. MINIX 3 resultó ser una plataforma de estudio adecuada, gracias a la disponibilidad de su código fuente, lo que facilitó la modificación y análisis de componentes internos reales.

Cada uno de los hitos planteados se completó satisfactoriamente:

- **Se personalizó el mensaje de bienvenida:** modificando el archivo `/etc/motd` y replicando el cambio en el código fuente, logrando que el mensaje personalizado apareciera al iniciar sesión.
- **Se depuró el bug en `pthread_mutex_trylock`:** identificando que la función se llamaba recursivamente a sí misma. La corrección consistió en reemplazar la llamada recursiva por `pthread_mutex_trylock`, resolviendo así el bloqueo del programa de prueba. El programa validó el comportamiento esperado, devolviendo 0 en la primera llamada y `EDEADLK` (código 11) en la segunda.
- **Se implementó el comando `tree`:** desarrollando un comando que recorre recursivamente el sistema de archivos utilizando llamadas al sistema (`opendir`, `readdir`, `closedir`, `stat`), aplicando indentación según el nivel de profundidad y evitando seguir enlaces simbólicos para prevenir ciclos.

- **Se modificó el planificador:** incorporando un contador de quantums por proceso e implementando una política de penalización por uso intensivo de CPU, basada en ventanas de tiempo. La validación experimental mostró una clara diferencia de tratamiento entre procesos CPU-bound (penalizados hasta prioridad 15) y procesos interactivos (sin penalización), cumpliendo con los criterios establecidos.

Gracias a este proyecto aprendimos en la práctica cómo funcionan la planificación por prioridades, la concurrencia con hilos y el acceso de bajo nivel al sistema de archivos. En definitiva, la experiencia adquirida resulta de gran valor para nuestra formación en la carrera, al integrar conceptos teóricos con la modificación práctica de un sistema operativo real.

5. Referencias consultadas

- Guerra, C. (2026). *Setup inicial de MINIX* [Video]. Grupo de Sistemas Operativos 25-26, Universidad de La Habana.

Utilizado como guía para instalar el sistema operativo y preparar el ambiente de trabajo

- MINIX 3. (2016). *Installing MINIX 3 . MINIX 3 Wiki*. Recuperado de <https://wiki.minix3.org/doku.php?id=install>
- MINIX 3. (2018). *Download MINIX 3 Wiki*. Recuperado de <https://wiki.minix3.org/doku.php?id=download>
- Colectivo de autores (2026). Conferencia #2 Introducción al Lenguaje C
Para la programación del comando tree y la modificación del scheduler tuvimos que emplear el lenguaje, el cual no conocíamos previamente.
- Colectivo de autores (2026). Conferencia #4 Control Y Planificación de Ejecuciones. Hilos
Para conocer los conceptos básicos sobre el manejo de hilos.
- Colectivo de autores (2026). Conferencia #5 Planificación
Para entender el funcionamiento de las diferentes estrategias de planificación de la CPU, como el Round Robin y el Multi-Level Feedback Queue (MLFQ).

6. Declaración de uso de IA

Fue utilizada como:

- Ayuda inicial para interpretar ciertos errores durante el proceso de instalación, como problemas de conectividad o de espacio para clonar y ejecutar el make.
- Asistencia para lograr conectar las terminales de minix y Ubuntu mediante SSH.
- Ayuda para conocer el uso exacto de varios comandos necesarios para la edición o supervisión de procesos en la terminal (cp, grep, top, etc, modo de compilar con cc, etc).

- Para mayor comprensión de la relación entre ‘pthread’ (capa de compatibilidad) y ‘mthread’ (implementación nativa de MINIX).
- Asistencia para resolver conflictos al hacer ‘git pull’ y ‘git push’, y para manejar el token de autenticación.
- En desconocimiento de qué funciones o macros podían utilizarse para tareas como comprobar si algo se trataba de un enlace simbólico, y entender que hacían, recibían y devolvían las funciones que trabajaban con los directorios, y la biblioteca que incluir.
- Para comprobar errores o la falta de ellos de sintaxis en c (ejemplo: falta un ;).
- Asistencia con la interfaz de virtualbox (ejemplo: como crear un snapshot de tu estado actual para salvaguardarlo).
- Para entender el código de las funciones originales de `do_no_quantum` y `balance_queues` y cuál era el archivo donde definía los campos de variable para rmp para poder definir `quantum_count`.
- Ayuda con la redacción de las referencias para este informe, siguiendo el formato APA, así como en la revisión de estilo y corrección de errores tipográficos en el resto del informe.